

Analysis

July 8, 2026

1 NGO Country Segmentation With Unsupervised Learning

A starting NGO wants to determine the best countries where it can offer aid. For this, the leader of the NGO has created a data collection team that has gathered some basic, preliminary information about different countries of the world. These metrics refer to a country's current state in terms of wealth and economic prosperity, health development, trade relationships, and other indicators. With this information, we're tasked with finding groups of countries that might require aid, which will then become the priority for this organization's directed efforts.

This notebook develops an end-to-end unsupervised learning pipeline using `Country-data.csv`. The workflow includes data exploration, preprocessing, feature engineering, PCA-based interpretation, clustering model comparison, final model selection, cluster profiling, robustness checks, and report-ready findings.

1.1 Objective

The objective of this report is to identify meaningful groups of countries based on socioeconomic, health, demographic, and macroeconomic indicators. The analysis is framed for an NGO that wants to screen countries by broad aid-priority profiles, with particular attention to countries showing high child mortality, low income, low GDP per capita, lower life expectancy, and other indicators of development need. We want to obtain a model that strikes a balance of good separation and offers good interpretability for us to be able to identify priority countries. Moreover, this analysis will also serve as a replicable pipeline to perform EDA and data cleanup, to then train a series of unsupervised learning models for this particular problem.

1.2 Setup

We set up our libraries for the analysis

```
[1]: import itertools
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.cluster.hierarchy import dendrogram, linkage
```

```

from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (
    calinski_harabasz_score,
    davies_bouldin_score,
    silhouette_score,
)
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
DATA_PATH = Path("Country-data.csv")
DICTIONARY_PATH = Path("data-dictionary.csv")

sns.set_theme(style="whitegrid", palette="colorblind")
pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", "{:.3f}".format)

```

1.3 Data Description

The dataset contains one row per country and a set of numeric indicators that describe economic capacity, health conditions, demographics, and trade exposure. The `country` field is an identifier used for labeling and interpretation and thus it is excluded from model training.

Variable roles in the analysis:

- **child_mort**: deaths of children under five per 1,000 live births; a direct indicator of humanitarian and health-system need.
- **exports**: exports as a percentage of GDP per capita; helps describe economic structure and external trade dependence.
- **health**: health spending as a percentage of GDP per capita; reflects the share of resources allocated to health.
- **imports**: imports as a percentage of GDP per capita; helps describe trade exposure and domestic supply dependence.
- **income**: net income per person; captures household-level economic capacity.
- **inflation**: annual GDP growth-rate inflation measure; signals macroeconomic instability when unusually high.
- **life_expec**: average life expectancy; summarizes broad health and development conditions.
- **total_fer**: fertility rate; often relates to development stage, healthcare access, and demographic pressure.
- **gdpp**: GDP per capita; captures national economic output per person.

Together, these variables allow the analysis to group countries by development profile and identify segments that may warrant different NGO strategies.

```

[2]: df = pd.read_csv(DATA_PATH)
      data_dictionary = pd.read_csv(DICTIONARY_PATH)

```

```

print(f"Dataset shape: {df.shape}")
display(df.head())

display(data_dictionary)

display(pd.DataFrame({
    "column": df.columns,
    "dtype": df.dtypes.astype(str).values,
    "missing_values": df.isna().sum().values,
    "unique_values": df.nunique().values,
}))

```

Dataset shape: (167, 10)

	country	child_mort	exports	health	imports	income \
0	Afghanistan	90.200	10.000	7.580	44.900	1610
1	Albania	16.600	28.000	6.550	48.600	9930
2	Algeria	27.300	38.400	4.170	31.400	12900
3	Angola	119.000	62.300	2.850	42.900	5900
4	Antigua and Barbuda	10.300	45.500	6.030	58.900	19100

	inflation	life_expec	total_fer	gdpp
0	9.440	56.200	5.820	553
1	4.490	76.300	1.650	4090
2	16.100	76.500	2.890	4460
3	22.400	60.100	6.160	3530
4	1.440	76.800	2.130	12200

	Column Name	Description
0	country	Name of the country
1	child_mort	Death of children under 5 years of age per 100...
2	exports	Exports of goods and services per capita. Give...
3	health	Total health spending per capita. Given as %ag...
4	imports	Imports of goods and services per capita. Give...
5	Income	Net income per person
6	Inflation	The measurement of the annual growth rate of t...
7	life_expec	The average number of years a new born child w...
8	total_fer	The number of children that would be born to e...
9	gdpp	The GDP per capita. Calculated as the Total GD...

	column	dtype	missing_values	unique_values
0	country	str	0	167
1	child_mort	float64	0	139
2	exports	float64	0	147
3	health	float64	0	147
4	imports	float64	0	151
5	income	int64	0	156
6	inflation	float64	0	156

7	life_expec	float64	0	127
8	total_fer	float64	0	138
9	gdpp	int64	0	157

1.4 Exploratory Data Analysis

This section checks quality, distributions, outliers, and correlations before modeling. These checks matter because clustering is sensitive to missing values, feature scales, skewed variables, and extreme observations.

```
[3]: print("Missing values by column:")
display(df.isna().sum().to_frame("missing_count"))

print(f"Duplicate rows: {df.duplicated().sum()}")
print(f"Duplicate country names: {df['country'].duplicated().sum()}")

display(df.describe().T)
```

Missing values by column:

	missing_count
country	0
child_mort	0
exports	0
health	0
imports	0
income	0
inflation	0
life_expec	0
total_fer	0
gdpp	0

Duplicate rows: 0

Duplicate country names: 0

	count	mean	std	min	25%	50%	75%	\
child_mort	167.000	38.270	40.329	2.600	8.250	19.300	62.100	
exports	167.000	41.109	27.412	0.109	23.800	35.000	51.350	
health	167.000	6.816	2.747	1.810	4.920	6.320	8.600	
imports	167.000	46.890	24.210	0.066	30.200	43.300	58.750	
income	167.000	17144.689	19278.068	609.000	3355.000	9960.000	22800.000	
inflation	167.000	7.782	10.571	-4.210	1.810	5.390	10.750	
life_expec	167.000	70.556	8.893	32.100	65.300	73.100	76.800	
total_fer	167.000	2.948	1.514	1.150	1.795	2.410	3.880	
gdpp	167.000	12964.156	18328.705	231.000	1330.000	4660.000	14050.000	

	max
child_mort	208.000
exports	200.000
health	17.900

```

imports      174.000
income       125000.000
inflation     104.000
life_expec    82.800
total_fer     7.490
gdpp          105000.000

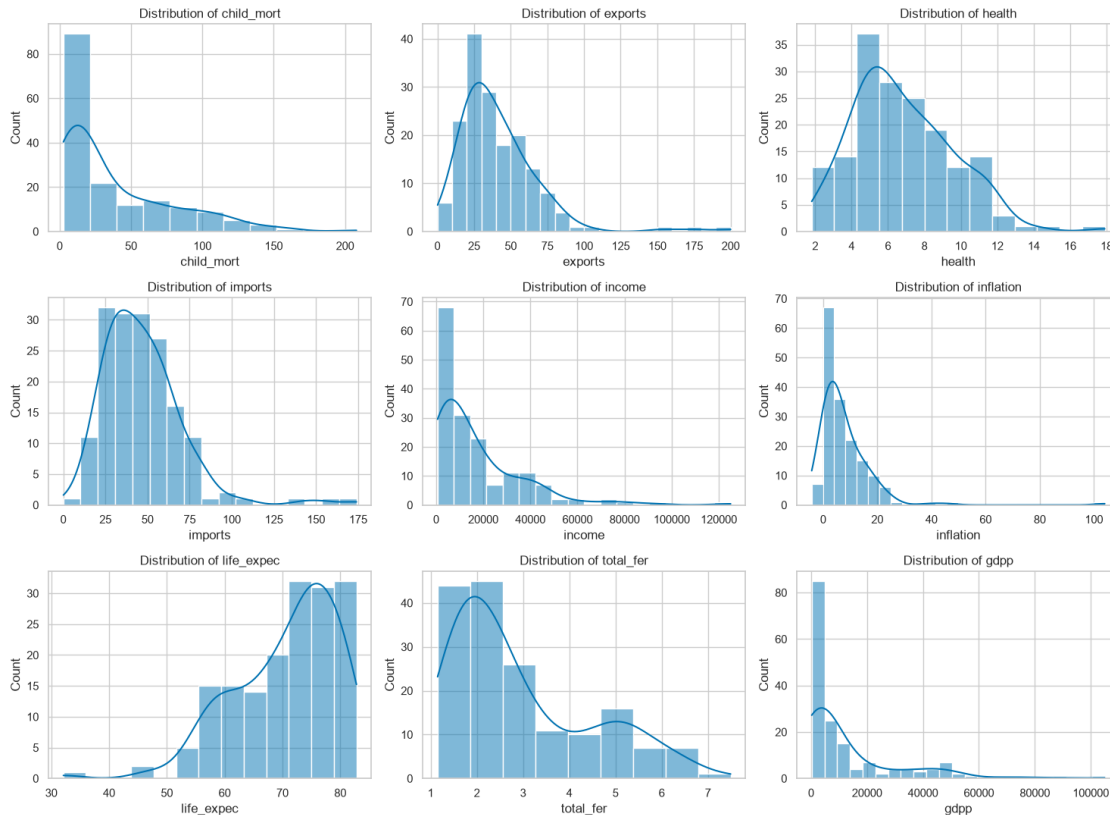
```

```

[4]: numeric_cols = df.drop(columns="country").columns.tolist()

fig, axes = plt.subplots(3, 3, figsize=(15, 11))
axes = axes.ravel()
for ax, col in zip(axes, numeric_cols):
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(f"Distribution of {col}")
plt.tight_layout()
plt.show()

```

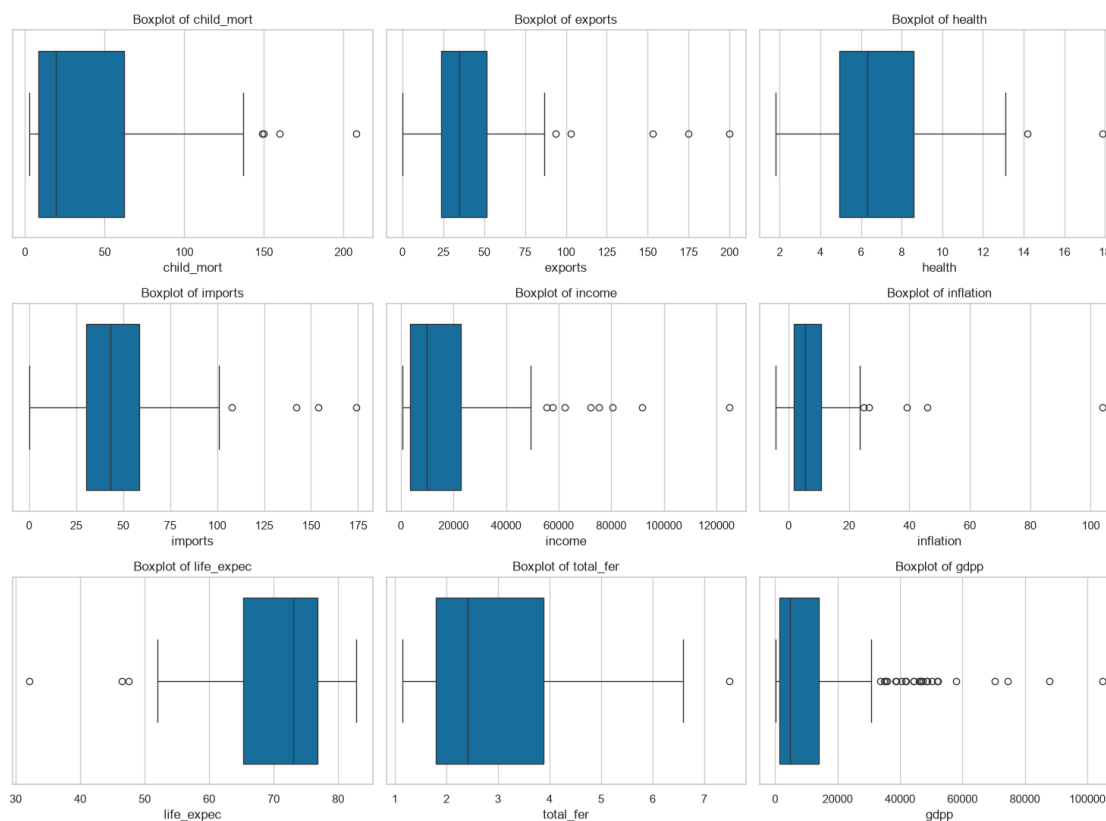


1.4.1 Data Analysis (I)

The histograms display certain characteristics of our feature variables that we must treat: - All of the features are skewed in some degree, whether positively or negatively skewed. Treatment and proper scaling of these variables is needed. - The scales are vastly different between variables, such

as `total_fer`, which is in units representing children and `gdpp`, which represents USD per capita. These will have to be scaled appropriately to prevent the models from giving more weight to the variables with a larger scale.

```
[5]: fig, axes = plt.subplots(3, 3, figsize=(15, 11))
      axes = axes.ravel()
      for ax, col in zip(axes, numeric_cols):
          sns.boxplot(x=df[col], ax=ax)
          ax.set_title(f"Boxplot of {col}")
      plt.tight_layout()
      plt.show()
```



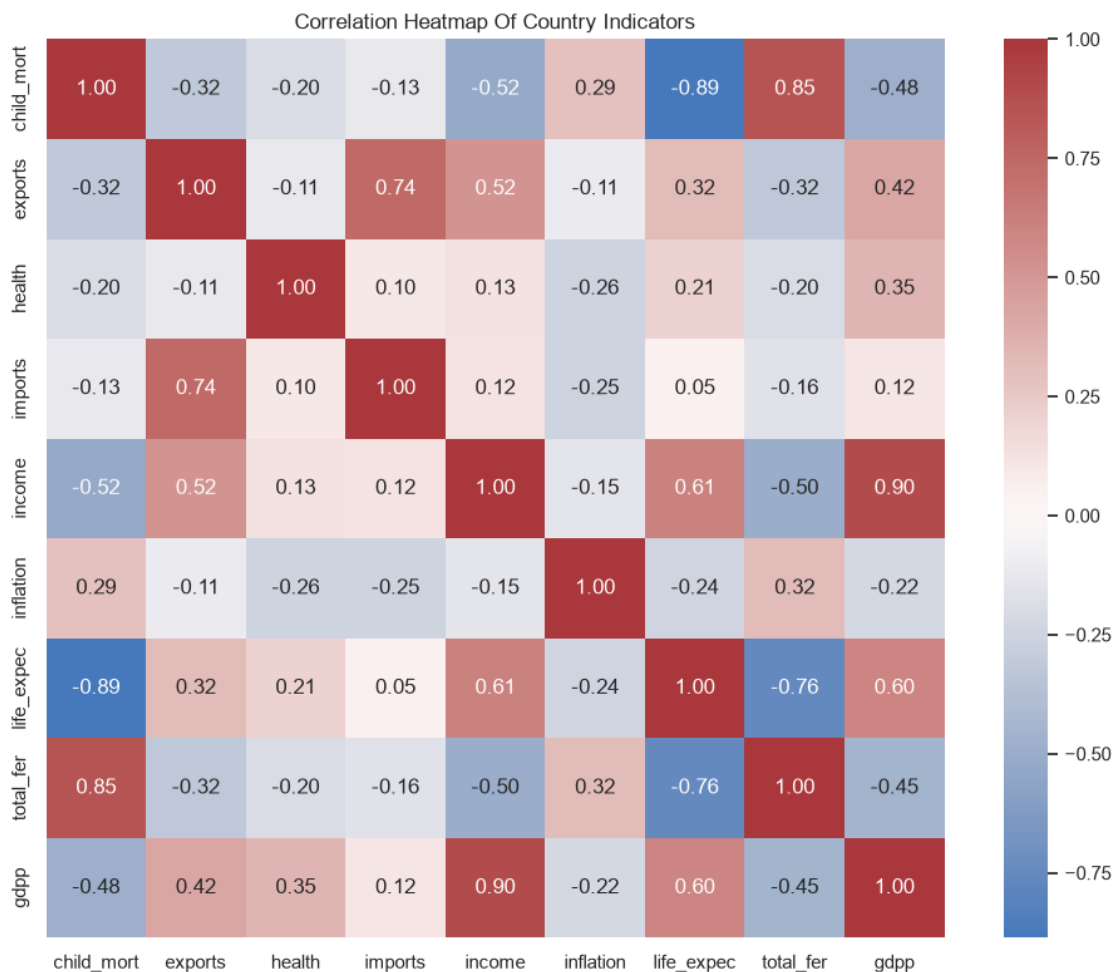
1.4.2 Data Analysis (II)

There are several outliers in many key indicators such as `gdpp`, `child_mort` or `income`. A good portion of these outliers lie at the upper extreme of the distributions, however, there are 2 main reasons as to why we may not want to delete these outliers: - The outlier may represent a valid signal. Some countries can present an unusually high child mortality rate while also presenting a GDP per capita that is considered an outlier. This can be due to the fact that a country may have recently suffered a natural disaster or has been in an armed conflict, while preserving a good economy through trade. This country would be a valid target for the aid of the NGO - Wealth inequality can make it so that income per capita and GDP per capita are both high, but they're

not strict indicators of the well-being of a country.

The main assumption behind these reasons is that direct income and GDP per capita don't necessarily equate to good living conditions in a country. One example of such a country is Dubai, a place that boasts a very high level of wealth and income. However, despite the luxurious image of the country, most of it is built upon lax labor laws towards migrants who want to make a living. The majority of the people living in Dubai are not really rich, and only the tourists and important figures visiting the place live in good conditions, whereas most of the workers responsible for the place's growth do not live as comfortably.

```
[6]: plt.figure(figsize=(10, 8))
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt=".2f", cmap="vlag",
            center=0, square=True)
plt.title("Correlation Heatmap Of Country Indicators")
plt.tight_layout()
plt.show()
```



1.4.3 Data Analysis (III)

- The main thread of the correlation heatmap is clear: the more income and the more wealth a country has, the better the other health and fertility indicators are as well. Factors such as `gdpp` are positively correlated with health or life expectancy, even if not meaningfully, while showing strong correlation with income and `life_expectancy`, as well as a negative correlation with child mortality, since a higher GDP per capita means that families have more resources to take care of their children.
- Total fertility rate is highly correlated to child mortality. This is usually because in high child mortality countries, families need to have more children in order for the family to grow. It is also negatively correlated heavily with life expectancy, as it means people live less time, thus giving more weight to the decision to have more children. This variable is negatively correlated to essentially all wealth indicators. In general, the wealthier a country is, the less children people tend to have, since there are enough resources to properly raise fewer children and for these children to survive into adulthood.
- Inflation shows weak negative correlation with positive economic indicators and weak positive correlation with less wealthy indicators.
- Child mortality is highly negatively correlated to life expectancy and very positively correlated to total fertility, for the reasons outlined above.

1.5 Feature Engineering And Preprocessing

In order to perform our analysis, three feature sets are prepared:

1. **Original scaled features:** the main modeling input, and the one we will prioritize most.
2. **Log-adjusted features:** a robustness check for skewed `income` and `gdpp`. This is created because they are very heavily skewed.
3. **Engineered value features:** a robustness check that converts percentage-based export, import, and health measures into approximate per-capita values. This can add more value to the person-by-person variables, and thus offer a clearer picture of the realities of individual family blocks in different countries.

All model inputs are scaled with `StandardScaler` because the indicators use very different units and magnitudes.

```
[7]: feature_cols = [  
    "child_mort",  
    "exports",  
    "health",  
    "imports",  
    "income",  
    "inflation",  
    "life_expec",  
    "total_fer",  
    "gdpp",  
]  
  
country_labels = df["country"].copy()  
X_original = df[feature_cols].copy()
```

```

X_log = X_original.copy()
X_log["income"] = np.log1p(X_log["income"])
X_log["gdpp"] = np.log1p(X_log["gdpp"])

X_engineered = X_original.copy()
X_engineered["exports_value"] = X_engineered["exports"] * X_engineered["gdpp"] /
↪ 100
X_engineered["imports_value"] = X_engineered["imports"] * X_engineered["gdpp"] /
↪ 100
X_engineered["health_value"] = X_engineered["health"] * X_engineered["gdpp"] /
↪ 100

feature_sets_raw = {
    "original": X_original,
    "log_adjusted": X_log,
    "engineered_values": X_engineered,
}

scaled_feature_sets = {}
scalers = {}
for name, X in feature_sets_raw.items():
    scaler = StandardScaler()
    scaled_feature_sets[name] = pd.DataFrame(
        scaler.fit_transform(X),
        columns=X.columns,
        index=df.index,
    )
    scalers[name] = scaler

X_scaled = scaled_feature_sets["original"]
print("Primary modeling matrix shape:", X_scaled.shape)
display(X_scaled.head())

```

Primary modeling matrix shape: (167, 9)

	child_mort	exports	health	imports	income	inflation	life_expec	\
0	1.292	-1.138	0.279	-0.082	-0.808	0.157	-1.619	
1	-0.539	-0.480	-0.097	0.071	-0.375	-0.312	0.648	
2	-0.273	-0.099	-0.966	-0.642	-0.221	0.789	0.670	
3	2.008	0.775	-1.448	-0.165	-0.585	1.387	-1.179	
4	-0.696	0.161	-0.287	0.498	0.102	-0.602	0.704	

	total_fer	gdpp
0	1.903	-0.679
1	-0.860	-0.486
2	-0.038	-0.465
3	2.128	-0.516

4 -0.542 -0.042

1.6 PCA Exploration

PCA helps summarize the main dimensions of variation and provides a two-dimensional view of countries for visualization. The clustering models are still trained and evaluated separately. PCA will be used exclusively for dimensionality reduction, visualization and interpretation. It can serve as good alternative to training the models on the entire dataset and thus avoid large computational times and potentially reducing the overfit of our models. As we can see in the graph below, with 5 principal components we can explain 95% of the variance, which is less than our 9 total features. The number of features is not particularly big for us to consider PCA this much, but it is a good step in the pipeline. We can also try to see, using 2 principal components, a 2D representation of the data points, which is particularly helpful for visualization and interpretation.

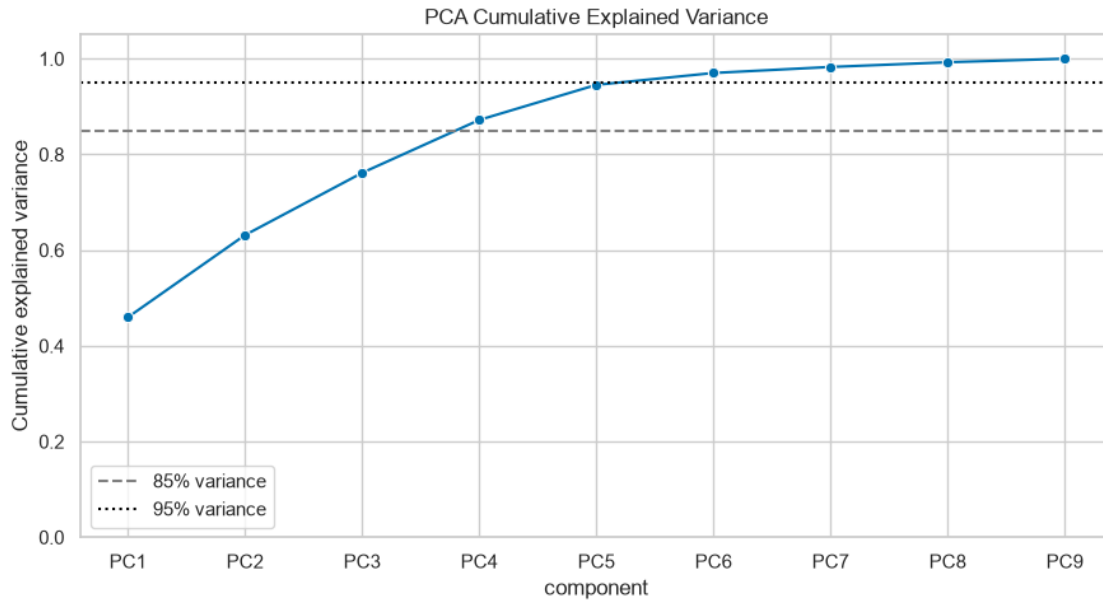
```
[8]: pca_full = PCA(random_state=RANDOM_STATE)
X_pca_full = pca_full.fit_transform(X_scaled)

pca_summary = pd.DataFrame({
    "component": [f"PC{i+1}" for i in range(len(pca_full.
↪explained_variance_ratio_))],
    "explained_variance_ratio": pca_full.explained_variance_ratio_,
    "cumulative_explained_variance": np.cumsum(pca_full.
↪explained_variance_ratio_),
})
display(pca_summary)

plt.figure(figsize=(9, 5))
sns.lineplot(
    data=pca_summary,
    x="component",
    y="cumulative_explained_variance",
    marker="o",
)
plt.axhline(0.85, color="gray", linestyle="--", label="85% variance")
plt.axhline(0.95, color="black", linestyle=":", label="95% variance")
plt.ylim(0, 1.05)
plt.title("PCA Cumulative Explained Variance")
plt.ylabel("Cumulative explained variance")
plt.legend()
plt.tight_layout()
plt.show()
```

	component	explained_variance_ratio	cumulative_explained_variance
0	PC1	0.460	0.460
1	PC2	0.172	0.631
2	PC3	0.130	0.761
3	PC4	0.111	0.872
4	PC5	0.073	0.945

5	PC6	0.025	0.970
6	PC7	0.013	0.983
7	PC8	0.010	0.993
8	PC9	0.007	1.000



```
[9]: pca_2 = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_2 = pd.DataFrame(
    pca_2.fit_transform(X_scaled),
    columns=["PC1", "PC2"],
    index=df.index,
)

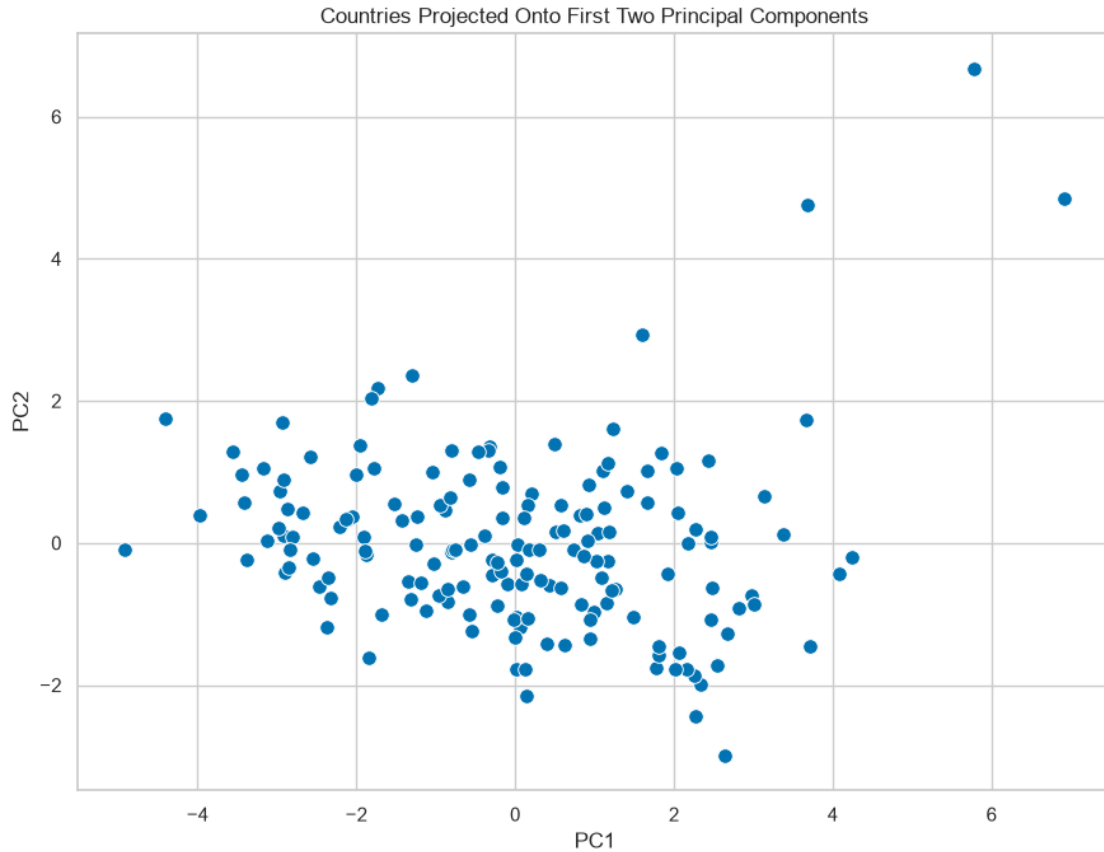
loadings = pd.DataFrame(
    pca_2.components_.T,
    columns=["PC1", "PC2"],
    index=feature_cols,
)

display(loadings.sort_values("PC1", key=abs, ascending=False))

plt.figure(figsize=(9, 7))
sns.scatterplot(data=X_pca_2, x="PC1", y="PC2", s=70)
plt.title("Countries Projected Onto First Two Principal Components")
plt.tight_layout()
plt.show()
```

	PC1	PC2
life_expec	0.426	-0.223
child_mort	-0.420	0.193

total_fer	-0.404	0.155
income	0.398	0.023
gdpp	0.393	-0.046
exports	0.284	0.613
inflation	-0.193	-0.008
imports	0.161	0.672
health	0.151	-0.243



1.7 Model Selection and Model Evaluation

We use three primary unsupervised clustering approaches:

- KMeans clustering: easy to explain due to high interpretability and very good model to act as baseline.
- Hierarchical Agglomerative Clustering: using dendograms, it can offer high interpretability and can find nested relationships inside country groups, offering an additional layer of segmentation and useful information as subgroups.
- Gaussian Mixture Models: less intuitive than KMeans, but offers greater flexibility because some countries can present mixed features, allowing us to capture more uncommon signals or outliers and treat them appropriately.

Other models have been considered, such as DBSCAN, Mean Shift, MDS and NMF, but they offer less interpretability than our selected models and they may not offer much improvement, potentially offering even worse results due to noise and the way our data points exist.

Because this is an unsupervised problem, there is no target variable. Model comparison uses internal validation metrics plus cluster-size balance and interpretability. Here are the definitions for some helper functions to evaluate our models.

For model evaluation we chose the following metrics: - Silhouette: captures how well a data point fits inside its cluster versus other clusters. It is useful for individual fit and cluster separation. - Calinski-Harabasz score: it compares how well clusters are separated using their compactness. - Davies-Bouldin score: measures average similarity between clusters. It can be a good measure of clarity when determining priority groups of countries.

```
[10]: def cluster_size_summary(labels):
    counts = pd.Series(labels).value_counts().sort_index()
    return {
        "min_cluster_size": int(counts.min()),
        "max_cluster_size": int(counts.max()),
        "cluster_sizes": dict(counts),
    }

def evaluate_labels(X, labels):
    labels = np.asarray(labels)
    n_clusters = len(np.unique(labels))
    if n_clusters < 2 or n_clusters >= len(labels):
        return {
            "silhouette": np.nan,
            "calinski_harabasz": np.nan,
            "davies_bouldin": np.nan,
        }
    return {
        "silhouette": silhouette_score(X, labels),
        "calinski_harabasz": calinski_harabasz_score(X, labels),
        "davies_bouldin": davies_bouldin_score(X, labels),
    }

def add_common_result(results, model_name, params, X, labels, extra=None):
    metrics = evaluate_labels(X, labels)
    sizes = cluster_size_summary(labels)
    row = {
        "model": model_name,
        **params,
        **metrics,
        "min_cluster_size": sizes["min_cluster_size"],
        "max_cluster_size": sizes["max_cluster_size"],
    }
```

```

        "cluster_sizes": sizes["cluster_sizes"],
    }
    if extra:
        row.update(extra)
    results.append(row)

```

1.8 KMeans Hyperparameter Tuning

KMeans is the baseline model because it produces simple, interpretable country segments. The main tuning parameter is the number of clusters.

```

[11]: kmeans_results = []
      kmeans_models = {}

      for n_clusters, init, n_init, max_iter in itertools.product(
          [2, 3, 4, 5, 6],
          ["k-means++", "random"],
          [10, 25, 50],
          [300, 500],
      ):
          model = KMeans(
              n_clusters=n_clusters,
              init=init,
              n_init=n_init,
              max_iter=max_iter,
              random_state=RANDOM_STATE,
          )
          labels = model.fit_predict(X_scaled)
          params = {
              "n_clusters": n_clusters,
              "init": init,
              "n_init": n_init,
              "max_iter": max_iter,
          }
          add_common_result(kmeans_results, "KMeans", params, X_scaled, labels,
↪{"inertia": model.inertia_})
          kmeans_models[(n_clusters, init, n_init, max_iter)] = model

      kmeans_results_df = pd.DataFrame(kmeans_results).sort_values(
          ["silhouette", "davies_bouldin"],
          ascending=[False, True],
      )
      display(kmeans_results_df.head(10))

```

File "C:\Users\adrian\AppData\Local\Python\pythoncore-3.12-64\Lib\site-packages\joblib\externals\loky\backend\context.py", line 247, in
_count_physical_cores
cpu_count_physical = _count_physical_cores_win32()

```

~~~~~
File "C:\Users\adrian\AppData\Local\Python\pythoncore-3.12-64\Lib\site-
packages\joblib\externals\loky\backend\context.py", line 299, in
_count_physical_cores_win32
    cpu_info = subprocess.run(
        ~~~~~

File
"C:\Users\adrian\AppData\Local\Python\pythoncore-3.12-64\Lib\subprocess.py",
line 548, in run
    with Popen(*popenargs, **kwargs) as process:
        ~~~~~

File
"C:\Users\adrian\AppData\Local\Python\pythoncore-3.12-64\Lib\subprocess.py",
line 1026, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
File
"C:\Users\adrian\AppData\Local\Python\pythoncore-3.12-64\Lib\subprocess.py",
line 1538, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
        ~~~~~

```

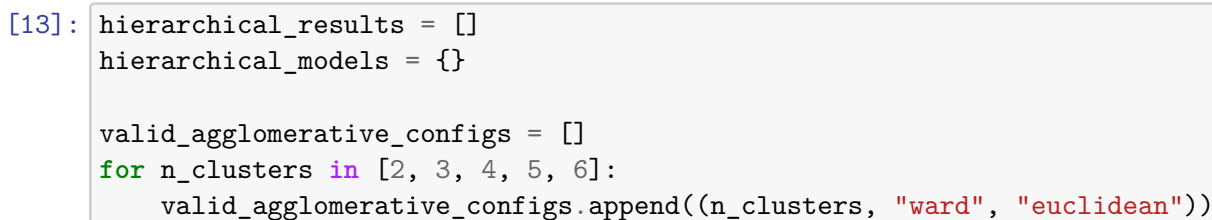
	model	n_clusters	init	n_init	max_iter	silhouette	\
30	KMeans	4	random	10	300	0.304	
31	KMeans	4	random	10	500	0.304	
26	KMeans	4	k-means++	25	300	0.302	
27	KMeans	4	k-means++	25	500	0.302	
28	KMeans	4	k-means++	50	300	0.302	
29	KMeans	4	k-means++	50	500	0.302	
32	KMeans	4	random	25	300	0.301	
33	KMeans	4	random	25	500	0.301	
34	KMeans	4	random	50	300	0.301	
35	KMeans	4	random	50	500	0.301	

	calinski_harabasz	davies_bouldin	min_cluster_size	max_cluster_size	\
30	62.211	1.042	3	87	
31	62.211	1.042	3	87	
26	62.263	1.041	3	88	
27	62.263	1.041	3	88	
28	62.263	1.041	3	88	
29	62.263	1.041	3	88	
32	62.274	1.044	3	87	
33	62.274	1.044	3	87	
34	62.274	1.044	3	87	
35	62.274	1.044	3	87	

	cluster_sizes	inertia
30	{0: 48, 1: 87, 2: 3, 3: 29}	700.703
31	{0: 48, 1: 87, 2: 3, 3: 29}	700.703

1.9 Hierarchical Agglomerative Clustering Hyperparameter Tuning

```
[12]: plt.figure(figsize=(14, 6))
ward_linkage = linkage(X_scaled, method="ward")
dendrogram(ward_linkage, labels=country_labels.values, leaf_rotation=90,
            leaf_font_size=7)
plt.title("Ward Hierarchical Clustering Dendrogram")
plt.xlabel("Country")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```



```

    for linkage_method, metric in itertools.product(["complete", "average"],
    ↪ ["euclidean", "manhattan", "cosine"]):
        valid_agglomerative_configs.append((n_clusters, linkage_method, metric))

for n_clusters, linkage_method, metric in valid_agglomerative_configs:
    model = AgglomerativeClustering(
        n_clusters=n_clusters,
        linkage=linkage_method,
        metric=metric,
    )
    labels = model.fit_predict(X_scaled)
    params = {
        "n_clusters": n_clusters,
        "linkage": linkage_method,
        "metric": metric,
    }
    add_common_result(hierarchical_results, "Hierarchical", params, X_scaled,
    ↪ labels)
    hierarchical_models[(n_clusters, linkage_method, metric)] = model

hierarchical_results_df = pd.DataFrame(hierarchical_results).sort_values(
    ["silhouette", "davies_bouldin"],
    ascending=[False, True],
)
display(hierarchical_results_df.head(10))

```

	model	n_clusters	linkage	metric	silhouette \
4	Hierarchical	2	average	euclidean	0.630
5	Hierarchical	2	average	manhattan	0.572
11	Hierarchical	3	average	euclidean	0.562
12	Hierarchical	3	average	manhattan	0.562
18	Hierarchical	4	average	euclidean	0.497
19	Hierarchical	4	average	manhattan	0.497
25	Hierarchical	5	average	euclidean	0.386
9	Hierarchical	3	complete	manhattan	0.326
26	Hierarchical	5	average	manhattan	0.316
0	Hierarchical	2	ward	euclidean	0.315

	calinski_harabasz	davies_bouldin	min_cluster_size	max_cluster_size \
4	11.391	0.264	1	166
5	23.141	0.648	3	164
11	18.870	0.513	1	163
12	18.870	0.513	1	163
18	15.305	0.470	1	162
19	15.305	0.470	1	162
25	13.195	0.452	1	161
9	43.213	1.019	3	132

26	36.642	0.694	1	117
0	51.883	1.327	34	133

	cluster_sizes
4	{0: 166, 1: 1}
5	{0: 164, 1: 3}
11	{0: 163, 1: 1, 2: 3}
12	{0: 163, 1: 3, 2: 1}
18	{0: 162, 1: 1, 2: 3, 3: 1}
19	{0: 162, 1: 3, 2: 1, 3: 1}
25	{0: 161, 1: 1, 2: 3, 3: 1, 4: 1}
9	{0: 132, 1: 32, 2: 3}
26	{0: 45, 1: 3, 2: 1, 3: 1, 4: 117}
0	{0: 133, 1: 34}

1.10 Gaussian Mixture Model Hyperparameter Tuning

Gaussian Mixture Models provide probabilistic cluster assignments and allow more flexible cluster shapes than KMeans. AIC and BIC are used because they are designed for mixture-model comparison.

```
[14]: gmm_results = []
      gmm_models = {}

      for n_components, covariance_type, n_init, max_iter in itertools.product(
          [2, 3, 4, 5, 6],
          ["full", "tied", "diag", "spherical"],
          [10, 25],
          [300, 500],
      ):
          model = GaussianMixture(
              n_components=n_components,
              covariance_type=covariance_type,
              n_init=n_init,
              max_iter=max_iter,
              random_state=RANDOM_STATE,
          )
          labels = model.fit_predict(X_scaled)
          params = {
              "n_components": n_components,
              "covariance_type": covariance_type,
              "n_init": n_init,
              "max_iter": max_iter,
          }
          add_common_result(
              gmm_results,
              "GMM",
              params,
```

```

        X_scaled,
        labels,
        {"bic": model.bic(X_scaled), "aic": model.aic(X_scaled)},
    )
    gmm_models[(n_components, covariance_type, n_init, max_iter)] = model

gmm_results_df = pd.DataFrame(gmm_results).sort_values(
    ["bic", "silhouette"],
    ascending=[True, False],
)
display(gmm_results_df.head(10))

```

	model	n_components	covariance_type	n_init	max_iter	silhouette	\
34	GMM	4	full	25	300	0.175	
35	GMM	4	full	25	500	0.175	
64	GMM	6	full	10	300	0.184	
65	GMM	6	full	10	500	0.184	
66	GMM	6	full	25	300	0.184	
67	GMM	6	full	25	500	0.184	
16	GMM	3	full	10	300	0.238	
17	GMM	3	full	10	500	0.238	
18	GMM	3	full	25	300	0.238	
19	GMM	3	full	25	500	0.238	

	calinski_harabasz	davies_bouldin	min_cluster_size	max_cluster_size	\
34	40.346	1.577		5	72
35	40.346	1.577		5	72
64	49.151	1.381		3	51
65	49.151	1.381		3	51
66	49.151	1.381		3	51
67	49.151	1.381		3	51
16	46.419	1.284		5	101
17	46.419	1.284		5	101
18	46.419	1.284		5	101
19	46.419	1.284		5	101

	cluster_sizes	bic	aic
34	{0: 5, 1: 54, 2: 72, 3: 36}	2333.466	1650.626
35	{0: 5, 1: 54, 2: 72, 3: 36}	2333.466	1650.626
64	{0: 51, 1: 30, 2: 37, 3: 3, 4: 40, 5: 6}	2343.752	1317.932
65	{0: 51, 1: 30, 2: 37, 3: 3, 4: 40, 5: 6}	2343.752	1317.932
66	{0: 51, 1: 30, 2: 37, 3: 3, 4: 40, 5: 6}	2343.752	1317.932
67	{0: 51, 1: 30, 2: 37, 3: 3, 4: 40, 5: 6}	2343.752	1317.932
16	{0: 61, 1: 101, 2: 5}	2429.337	1917.986
17	{0: 61, 1: 101, 2: 5}	2429.337	1917.986
18	{0: 61, 1: 101, 2: 5}	2429.337	1917.986
19	{0: 61, 1: 101, 2: 5}	2429.337	1917.986

1.11 Model Comparison

The best candidates from each model family are compared side by side in the following table and with the following graphs. We use a 2D representation using the first 2 PCA components to get a better idea of how the clusters have been formed, even if the models have fit the 9 feature dataset.

```
[15]: best_kmeans = kmeans_results_df.iloc[0]
best_hierarchical = hierarchical_results_df.iloc[0]
best_gmm = gmm_results_df.iloc[0]

comparison_cols = [
    "model",
    "n_clusters",
    "n_components",
    "init",
    "linkage",
    "metric",
    "covariance_type",
    "silhouette",
    "calinski_harabasz",
    "davies_bouldin",
    "bic",
    "aic",
    "min_cluster_size",
    "max_cluster_size",
    "cluster_sizes",
]

model_comparison = pd.DataFrame([best_kmeans, best_hierarchical, best_gmm])
for col in comparison_cols:
    if col not in model_comparison.columns:
        model_comparison[col] = np.nan
model_comparison = model_comparison[comparison_cols]
display(model_comparison)
```

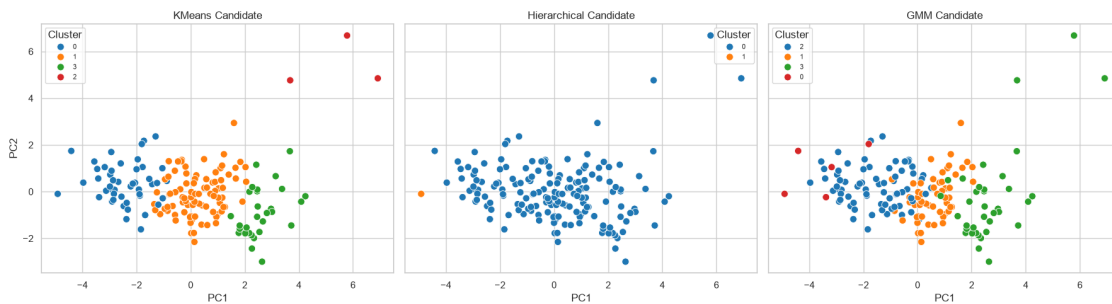
	model	n_clusters	n_components	init	linkage	metric	\
30	KMeans	4.000	NaN	random	NaN	NaN	
4	Hierarchical	2.000	NaN	NaN	average	euclidean	
34	GMM	NaN	4.000	NaN	NaN	NaN	

	covariance_type	silhouette	calinski_harabasz	davies_bouldin	bic	\
30	NaN	0.304	62.211	1.042	NaN	
4	NaN	0.630	11.391	0.264	NaN	
34	full	0.175	40.346	1.577	2333.466	

	aic	min_cluster_size	max_cluster_size	cluster_sizes
30	NaN	3	87	{0: 48, 1: 87, 2: 3, 3: 29}
4	NaN	1	166	{0: 166, 1: 1}
34	1650.626	5	72	{0: 5, 1: 54, 2: 72, 3: 36}

```
[16]: def labels_for_best_row(row):
    if row["model"] == "KMeans":
        key = (int(row["n_clusters"]), row["init"], int(row["n_init"]),
        ↪int(row["max_iter"]))
        return kmeans_models[key].labels_
    if row["model"] == "Hierarchical":
        key = (int(row["n_clusters"]), row["linkage"], row["metric"])
        return hierarchical_models[key].fit_predict(X_scaled)
    if row["model"] == "GMM":
        key = (int(row["n_components"]), row["covariance_type"],
        ↪int(row["n_init"]), int(row["max_iter"]))
        return gmm_models[key].predict(X_scaled)
    raise ValueError("Unknown model")

candidate_rows = [best_kmeans, best_hierarchical, best_gmm]
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharex=True, sharey=True)
for ax, row in zip(axes, candidate_rows):
    labels = labels_for_best_row(row)
    plot_df = X_pca_2.assign(cluster=labels.astype(str))
    sns.scatterplot(data=plot_df, x="PC1", y="PC2", hue="cluster", ax=ax, s=65,
    ↪palette="tab10")
    ax.set_title(f"{row['model']} Candidate")
    ax.legend(title="Cluster", fontsize=8)
plt.tight_layout()
plt.show()
```



We notice that the clusters are relatively hard to differentiate, as countries in our original dataset are spread across a continuum representing a gradual transition from a country with poor income, health and development indicators to a country with greater resources and better indicators, which can create overlapping clusters. Moreover, some information is lost when graphically representing PCA components, as the first 2 components explain around 80% of the variance.

We can see that even with these limitations, KMeans outperforms the other models heavily, showing the most natural clusters out of the models, and HCA offering a very poor approximation, where a single data point is in its own cluster and the rest are in their own cluster, thus offering no helpful information whatsoever. GMM comes closer, but some points chosen as belonging to their own

cluster do not represent clear outliers or warrants the separation despite the lower scores.

1.12 Final Model Selection

Thus, based on the performance reviews of the models and taking into account their interpretability, we choose KMeans with 3 to 4 clusters, since it offers both better interpretability and performance. The criteria for this choice are as follows:

1. We prefer 3 or 4 clusters because these map naturally to aid-priority segments across the continuum of countries.
2. Among those candidates, prefer strong silhouette and Davies-Bouldin scores, that is, countries that are packed together clearly and form meaningful clusters.
3. We avoid solutions with very small clusters unless they reveal a clearly meaningful country segment.

```
[17]: interpretable_kmeans = kmeans_results_df[kmeans_results_df["n_clusters"]
      ↪isin([3, 4])].copy()
final_row = interpretable_kmeans.sort_values(
    ["silhouette", "davies_bouldin"],
    ascending=[False, True],
).iloc[0]

final_key = (
    int(final_row["n_clusters"]),
    final_row["init"],
    int(final_row["n_init"]),
    int(final_row["max_iter"]),
)
final_model = kmeans_models[final_key]
final_labels_raw = final_model.labels_

print("Selected final model parameters:")
display(final_row.to_frame("value"))
```

Selected final model parameters:

	value
model	KMeans
n_clusters	4
init	random
n_init	10
max_iter	300
silhouette	0.304
calinski_harabasz	62.211
davies_bouldin	1.042
min_cluster_size	3
max_cluster_size	87
cluster_sizes	{0: 48, 1: 87, 2: 3, 3: 29}
inertia	700.703

1.13 Final Cluster Profiling And Aid-Priority Labels

Clusters are profiled using the original unscaled indicators so the results can be interpreted in real-world terms.

```
[18]: clustered_df = df.copy()
clustered_df["cluster_raw"] = final_labels_raw

cluster_profile = clustered_df.groupby("cluster_raw")[feature_cols].
    .agg(["mean", "median"])
display(cluster_profile)

cluster_means = clustered_df.groupby("cluster_raw")[feature_cols].mean()
cluster_medians = clustered_df.groupby("cluster_raw")[feature_cols].median()
display(cluster_means)
```

	child_mort		exports		health		imports		
	mean	median	mean	median	mean	median	mean	median	\
cluster_raw									
0	92.367	89.750	28.546	23.800	6.296	5.480	41.443	39.750	
1	20.697	17.400	41.857	38.400	6.293	6.000	48.569	49.200	
2	4.133	2.800	176.000	175.000	6.793	7.770	156.667	154.000	
3	4.983	4.200	45.703	42.300	9.246	9.540	39.514	32.900	

	income		inflation		life_expec		total_fer	
	mean	median	mean	median	mean	median	mean	\
cluster_raw								
0	3937.771	1900.000	11.916	8.855	59.346	59.800	4.954	
1	13275.287	10900.000	7.369	5.710	73.089	73.900	2.273	
2	64033.333	72100.000	2.468	3.620	81.433	81.300	1.380	
3	45762.069	40700.000	2.728	1.160	80.386	80.400	1.815	

	gdpp		
	median	mean	median
cluster_raw			
0	5.055	1902.917	932.000
1	2.170	7161.770	5080.000
2	1.360	57566.667	46600.000
3	1.870	44065.517	41900.000

	child_mort	exports	health	imports	income	inflation	\
cluster_raw							
0	92.367	28.546	6.296	41.443	3937.771	11.916	
1	20.697	41.857	6.293	48.569	13275.287	7.369	
2	4.133	176.000	6.793	156.667	64033.333	2.468	
3	4.983	45.703	9.246	39.514	45762.069	2.728	

	life_expec	total_fer	gdpp
cluster_raw			

0	59.346	4.954	1902.917
1	73.089	2.273	7161.770
2	81.433	1.380	57566.667
3	80.386	1.815	44065.517

```
[19]: priority_score = (
    cluster_means["child_mort"].rank(ascending=True)
    + cluster_means["total_fer"].rank(ascending=True)
    + cluster_means["inflation"].rank(ascending=True)
    + cluster_means["income"].rank(ascending=False)
    + cluster_means["gdpp"].rank(ascending=False)
    + cluster_means["life_expec"].rank(ascending=False)
)

priority_order = priority_score.sort_values(ascending=False).index.tolist()
base_names = [
    "High Aid Priority",
    "Moderate Aid Priority",
    "Lower Aid Priority",
    "Specialized Review Group",
    "Lowest Aid Priority",
    "Additional Segment",
]

cluster_name_map = {cluster: base_names[i] for i, cluster in
    ↪enumerate(priority_order)}

clustered_df["aid_priority_segment"] = clustered_df["cluster_raw"].
    ↪map(cluster_name_map)
clustered_df["cluster"] = clustered_df["aid_priority_segment"]

print("Cluster naming map:")
display(pd.DataFrame({
    "cluster_raw": list(cluster_name_map.keys()),
    "aid_priority_segment": list(cluster_name_map.values()),
}).sort_values("cluster_raw"))

display(clustered_df[["country", "cluster_raw", "aid_priority_segment"] +
    ↪feature_cols].sort_values(["aid_priority_segment", "country"]))
```

Cluster naming map:

	cluster_raw	aid_priority_segment
0	0	High Aid Priority
1	1	Moderate Aid Priority
3	2	Specialized Review Group
2	3	Lower Aid Priority

	country	cluster_raw	aid_priority_segment	child_mort	exports	\
0	Afghanistan	0	High Aid Priority	90.200	10.000	

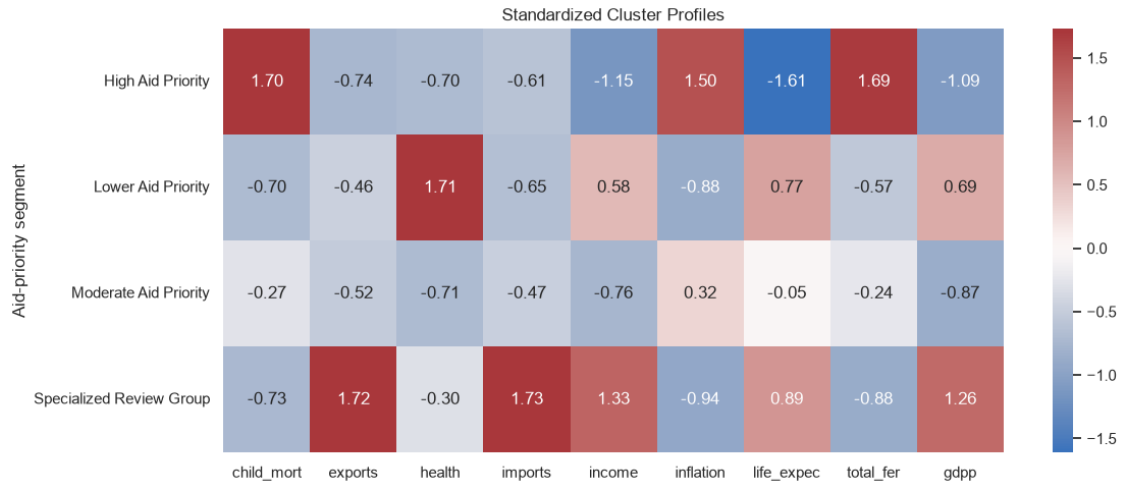
3	Angola	0	High Aid Priority	119.000	62.300
17	Benin	0	High Aid Priority	111.000	23.800
21	Botswana	0	High Aid Priority	52.500	43.600
25	Burkina Faso	0	High Aid Priority	116.000	19.200
..	...	...	...	...	...
163	Venezuela	1	Moderate Aid Priority	17.100	28.500
164	Vietnam	1	Moderate Aid Priority	23.300	72.000
91	Luxembourg	2	Specialized Review Group	2.800	175.000
98	Malta	2	Specialized Review Group	6.800	153.000
133	Singapore	2	Specialized Review Group	2.800	200.000

	health	imports	income	inflation	life_expec	total_fer	gdpp
0	7.580	44.900	1610	9.440	56.200	5.820	553
3	2.850	42.900	5900	22.400	60.100	6.160	3530
17	4.100	37.200	1820	0.885	61.800	5.360	758
21	8.300	51.300	13300	8.920	57.100	2.880	6350
25	6.740	29.600	1430	6.810	57.900	5.870	575
..	...	...	...	...	...	...	...
163	4.910	17.600	16500	45.900	75.400	2.470	13500
164	6.840	80.200	4490	12.100	73.100	1.950	1310
91	7.770	142.000	91700	3.620	81.300	1.630	105000
98	8.650	154.000	28300	3.830	80.300	1.360	21100
133	3.960	174.000	72100	-0.046	82.700	1.150	46600

[167 rows x 12 columns]

```
[20]: profile_for_plot = clustered_df.groupby("aid_priority_segment")[feature_cols].
      <-mean()
profile_scaled = pd.DataFrame(
    StandardScaler().fit_transform(profile_for_plot),
    index=profile_for_plot.index,
    columns=profile_for_plot.columns,
)

plt.figure(figsize=(12, 5))
sns.heatmap(profile_scaled, annot=True, fmt=".2f", cmap="vlag", center=0)
plt.title("Standardized Cluster Profiles")
plt.ylabel("Aid-priority segment")
plt.tight_layout()
plt.show()
```



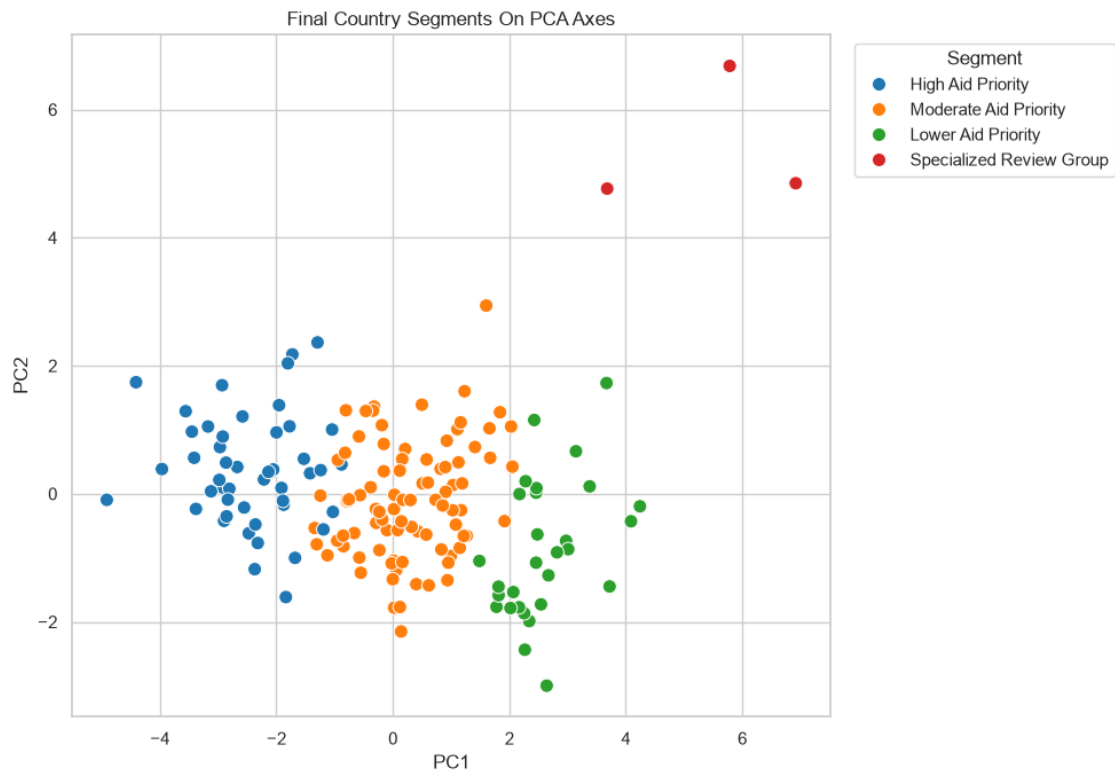
1.13.1 Final Model Interpretation and Profiled Groups

We can see that once we profile them, the countries in the High Priority group are quite cleanly identified by low development indicators. They present a high child mortality, high inflation, very low life expectancy, very high total fertility, and also the lowest GDP and income per capita. The Lower and Moderate aid groups offer clear improvements over these metrics, exceptuating Low Aid, which has a higher health investment in health than the rest despite the lower income and GDP they offer. The Specialized Review group was profiled like this because they represented the extreme outliers. However, upon close inspection, the reason why they're outliers is simply because they are the most developed countries out of all the dataset, and thus they do not really need aid from our NGO. We confirm the findings outlined in this heatmap by presenting the clusters in the 2D space using PCA components, and also bar charts for each of the features by grouping. Said graphs can be found below.

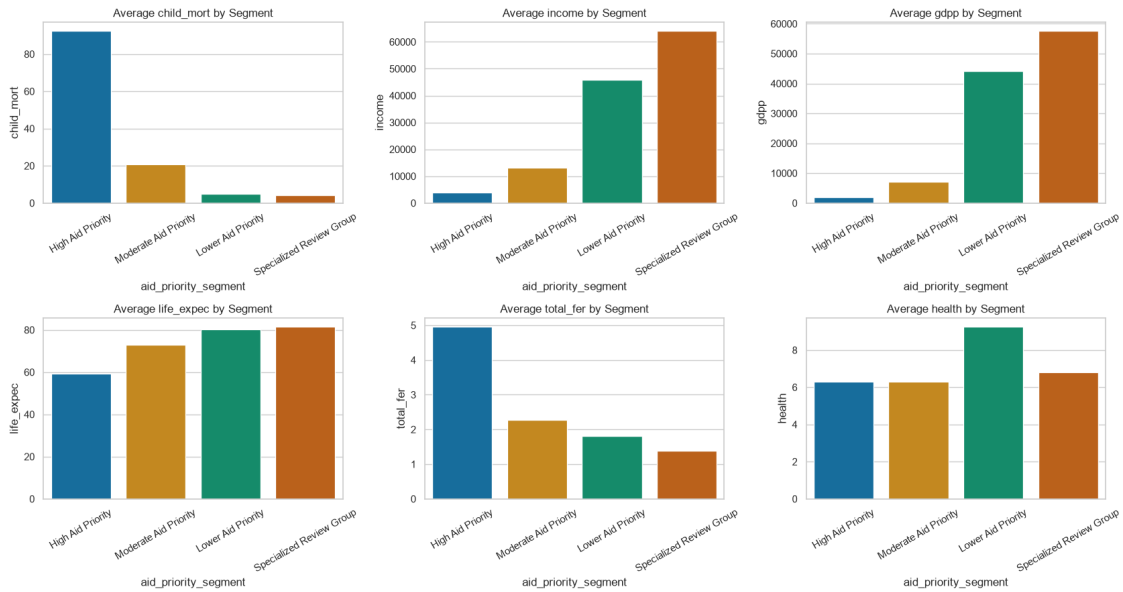
```
[21]: plot_df = X_pca_2.copy()
plot_df["aid_priority_segment"] = clustered_df["aid_priority_segment"]
plot_df["country"] = clustered_df["country"]

plt.figure(figsize=(10, 7))
sns.scatterplot(
    data=plot_df,
    x="PC1",
    y="PC2",
    hue="aid_priority_segment",
    s=75,
    palette="tab10",
)
plt.title("Final Country Segments On PCA Axes")
plt.legend(title="Segment", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
```

```
plt.show()
```



```
[22]: key_indicators = ["child_mort", "income", "gdp", "life_expec", "total_fer", "health"]
fig, axes = plt.subplots(2, 3, figsize=(17, 9))
axes = axes.ravel()
for ax, col in zip(axes, key_indicators):
    sns.barplot(
        data=clustered_df,
        x="aid_priority_segment",
        y=col,
        estimator="mean",
        errorbar=None,
        ax=ax,
        palette="colorblind",
    )
    ax.set_title(f"Average {col} by Segment")
    ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()
```



```
[23]: high_priority = clustered_df[clustered_df["aid_priority_segment"] == "High Aid_
    ↪Priority"].sort_values(
        ["child_mort", "income", "gdpp"],
        ascending=[False, True, True],
    )

    print(f"Countries in the High Aid Priority segment: {len(high_priority)}")
    display(high_priority[["country", "child_mort", "income", "gdpp", "life_expec",
    ↪"total_fer", "health"]])
```

Countries in the High Aid Priority segment: 48

	country	child_mort	income	gdpp	life_expec	\
66	Haiti	208.000	1500	662	32.100	
132	Sierra Leone	160.000	1220	399	55.000	
32	Chad	150.000	1930	897	56.500	
31	Central African Republic	149.000	888	446	47.500	
97	Mali	137.000	1870	708	59.500	
113	Nigeria	130.000	5150	2330	60.500	
112	Niger	123.000	814	348	58.800	
3	Angola	119.000	5900	3530	60.100	
37	Congo, Dem. Rep.	116.000	609	334	57.500	
25	Burkina Faso	116.000	1430	575	57.900	
64	Guinea-Bissau	114.000	1390	547	55.600	
17	Benin	111.000	1820	758	61.800	
40	Cote d'Ivoire	111.000	2690	1220	56.300	
49	Equatorial Guinea	111.000	33700	17100	60.900	
63	Guinea	109.000	1190	648	58.000	

28	Cameroon	108.000	2660	1310	57.300
106	Mozambique	101.000	918	419	54.500
87	Lesotho	99.700	2380	1170	46.500
99	Mauritania	97.400	3320	1200	68.200
26	Burundi	93.600	764	231	57.700
116	Pakistan	92.100	4280	1040	65.300
94	Malawi	90.500	1030	459	53.100
150	Togo	90.300	1210	488	58.700
0	Afghanistan	90.200	1610	553	56.200
88	Liberia	89.300	700	327	60.800
36	Comoros	88.200	1410	769	65.900
166	Zambia	83.100	3280	1460	52.000
155	Uganda	81.000	1540	595	56.800
56	Gambia	80.300	1660	562	65.500
84	Lao	78.900	3980	1140	63.800
142	Sudan	76.700	3370	1480	66.300
59	Ghana	74.700	3060	1310	62.200
147	Tanzania	71.900	2090	702	59.300
129	Senegal	66.800	2180	1000	64.000
107	Myanmar	64.400	3720	988	66.800
38	Congo, Rep.	63.900	5190	2740	60.400
55	Gabon	63.700	15400	8750	62.900
126	Rwanda	63.600	1350	563	64.600
81	Kiribati	62.700	1730	1490	60.700
149	Timor-Leste	62.600	1850	3600	71.100
93	Madagascar	62.200	1390	413	60.800
80	Kenya	62.200	2480	967	62.800
165	Yemen	56.300	4480	1310	67.500
108	Namibia	56.000	8460	5190	58.600
50	Eritrea	55.200	1420	482	61.700
137	South Africa	53.700	12000	7280	54.300
21	Botswana	52.500	13300	6350	57.100
72	Iraq	36.900	12700	4500	67.200

	total_fer	health
66	3.330	6.910
132	5.200	13.100
32	6.590	4.530
31	5.210	3.980
97	6.550	4.980
113	5.840	5.070
112	7.490	5.160
3	6.160	2.850
37	6.540	7.910
25	5.870	6.740
64	5.050	8.500
17	5.360	4.100
40	5.270	5.300

49	5.210	4.480
63	5.340	4.930
28	5.110	5.130
106	5.560	5.210
87	3.300	11.100
99	4.980	4.410
26	6.260	11.600
116	3.850	2.200
94	5.310	6.590
150	4.870	7.650
0	5.820	7.580
88	5.020	11.800
36	4.750	4.510
166	5.400	5.890
155	6.150	9.010
56	5.710	5.690
84	3.150	4.470
142	4.880	6.320
59	4.270	5.220
147	5.430	6.010
129	5.060	5.660
107	2.410	1.970
38	4.950	2.460
55	4.080	3.500
126	4.510	10.500
81	3.840	11.300
149	6.230	9.120
93	4.600	3.770
80	4.370	4.750
165	4.670	5.180
108	3.600	6.780
50	4.610	2.660
137	2.590	8.940
21	2.880	8.300
72	4.560	8.410

1.14 Robustness Checks

The final model choice is checked against alternative preprocessing choices. The goal of this segment is not to replace the final analysis, but to see whether the broad segmentation remains plausible when skewness and engineered value features are handled differently with the alternative datasets we have created in the EDA step.

```
[24]: def best_kmeans_for_matrix(X, allowed_clusters=(3, 4)):
      rows = []
      models = {}
      for n_clusters, init, n_init in itertools.product(allowed_clusters,
↳ ["k-means++"], [25, 50]):
```

```

        model = KMeans(
            n_clusters=n_clusters,
            init=init,
            n_init=n_init,
            max_iter=500,
            random_state=RANDOM_STATE,
        )
        labels = model.fit_predict(X)
        row = {
            "n_clusters": n_clusters,
            "init": init,
            "n_init": n_init,
            "max_iter": 500,
            **evaluate_labels(X, labels),
            **cluster_size_summary(labels),
        }
        rows.append(row)
        models[(n_clusters, init, n_init, 500)] = (model, labels)
        result_df = pd.DataFrame(rows).sort_values(["silhouette",
↪ "davies_bouldin"], ascending=[False, True])
        best = result_df.iloc[0]
        key = (int(best["n_clusters"]), best["init"], int(best["n_init"]),
↪ int(best["max_iter"]))
        return best, models[key][1]

robustness_rows = []
robustness_labels = {}

for feature_set_name, X_variant in scaled_feature_sets.items():
    best, labels = best_kmeans_for_matrix(X_variant)
    robustness_rows.append({"feature_set": feature_set_name, **best.to_dict()})
    robustness_labels[feature_set_name] = labels

n_components_85 = int(np.argmax(np.cumsum(pca_full.explained_variance_ratio_)
↪ >= 0.85) + 1)
X_pca_85 = pd.DataFrame(
    PCA(n_components=n_components_85, random_state=RANDOM_STATE).
↪ fit_transform(X_scaled),
    index=df.index,
)
best_pca, labels_pca = best_kmeans_for_matrix(X_pca_85)
robustness_rows.append({"feature_set": f"pca_{n_components_85}_components",
↪ **best_pca.to_dict()})
robustness_labels[f"pca_{n_components_85}_components"] = labels_pca

robustness_df = pd.DataFrame(robustness_rows)
display(robustness_df[[

```

```

    "feature_set",
    "n_clusters",
    "silhouette",
    "calinski_harabasz",
    "davies_bouldin",
    "min_cluster_size",
    "max_cluster_size",
    "cluster_sizes",
  ])

```

	feature_set	n_clusters	silhouette	calinski_harabasz	\
0	original	4	0.302	62.263	
1	log_adjusted	4	0.254	67.323	
2	engineered_values	4	0.317	75.401	
3	pca_4_components	4	0.355	84.144	

	davies_bouldin	min_cluster_size	max_cluster_size	\
0	1.041	3	88	
1	1.148	3	64	
2	1.009	2	88	
3	0.904	3	87	

	cluster_sizes
0	{0: 88, 1: 3, 2: 46, 3: 30}
1	{0: 3, 1: 64, 2: 62, 3: 38}
2	{0: 47, 1: 30, 2: 88, 3: 2}
3	{0: 87, 1: 3, 2: 48, 3: 29}

While the 4 PCA dataset may offer relatively better scores and better performance, the main drawback is interpretability. Even if the clusters are more cleanly defined, it is much more difficult or impossible to articulate exactly what each of the 4 PCA components represent. Thus, for our stakeholders, we opted not to include it, but this check serves as a good step in the pipeline if our objective was to simply offer better clustering without the need for good interpretability. In that case, trading off the interpretability could be worth it.

1.15 Key Findings

1. The countries separate meaningfully by broad development and health conditions. Our final model finds 4 meaningful clusters, and clearly identifies a High Priority Aid group for our NGO to focus on.
2. The highest-priority segment shows high child mortality, low income, low GDP per capita, lower life expectancy, and high fertility, key indicators for a country in need of humanitarian aid.
3. PCA helps show that much of the variation is driven by a general development axis, and with 4-5 key components we can explain 95% of the variance in the data.
4. KMeans provides a transparent segmentation that is easier for NGO stakeholders to interpret than more complex alternatives, while simultaneously providing the best and most clear performance.

1.16 Next Steps

Our best selected model successfully found a High Priority group that our NGO can focus on aiding. We outline some next steps:

- Countries requiring immediate review: Haiti, Sierra Leone, Chad, Central African Republic and Mali are the top 5 countries in need of aid. They show high fertility rate, high child mortality rate, combined with a low income, low health investment and low gdp per capita.
- Additional data needed before action: there is little information about conflict, food security, governance, climate risk, or subnational data, and this information is critical before adopting any further strategy.

Our recommendation is to collect additional data and analyze each country's specific context and needs. After this additional profiling, it is then important to develop a step-by-step action plan focusing on aid in a certain weakness of the country, such as health, or childhood mortality, or economic relief.

1.17 16. Limitations And Future Work

This analysis has several limitations:

- The data is cross-sectional and does not show whether countries are improving or deteriorating.
- Country-level averages hide regional inequality and vulnerable subpopulations.
- The dataset lacks direct measures of poverty, conflict, governance, education, food security, climate risk, and NGO operating constraints.
- Several indicators are correlated, so clustering may mainly reflect a broad development-level dimension.
- Clustering validation is imperfect because there are no true labels.
- KMeans assumes compact clusters, hierarchical clustering depends on linkage and distance choices, and GMM assumes mixture distributions that may not fully represent socioeconomic patterns.
- PCA is useful for visualization but can oversimplify country differences.
- The results should support strategic prioritization and further investigation, not final funding decisions.

Future work should incorporate more recent data, time-series indicators, subnational measures, and operational variables relevant to NGO planning.